

License Plate Recognition using ESP32-CAM

Mohammed Sheik Syed T, Mukesh Raj K, Seyed Ryyan
Department of Electronics and Communication Engineering
B.S. Abdur Rahman Crescent Institute of Science and Technology
Vandalur, Chennai, India

Emails: 230051601071@crescent.education, 230051601079@crescent.education, 230051601101@crescent.education

Abstract—This project presents the design and implementation of an IoT-based License Plate Recognition (LPR) system using the ESP32-CAM microcontroller. The system captures images of vehicle license plates and performs edge-based image processing and Optical Character Recognition (OCR) using the OpenALPR API. By integrating the ESP32-CAM with Wi-Fi and cloud-based recognition services, the project demonstrates an efficient, low-cost solution for automated vehicle identification. The proposed system operates without additional computing devices, utilizing the onboard camera of the ESP32 for image capture and HTTP-based communication for cloud inference. Results show accurate detection and recognition under varied lighting conditions, proving its feasibility for real-time smart parking and security applications.

Index Terms—ESP32-CAM, License Plate Recognition, IoT, OpenALPR, Image Processing, Edge AI, Cloud API, Vehicle Monitoring.

I. INTRODUCTION

With the rapid growth of smart city infrastructures, automated vehicle monitoring has become essential for traffic management, security, and parking automation. Traditional license plate recognition systems rely on powerful computers or servers to process camera footage, making them costly and power-hungry.

The emergence of the ESP32-CAM microcontroller has introduced a powerful, low-cost, and Wi-Fi-enabled alternative for image acquisition and transmission. This module integrates an OV2640 camera sensor and supports direct internet connectivity, allowing embedded image capture and cloud-based processing.

This project focuses on building a **License Plate Recognition (LPR) system** using the ESP32-CAM and cloud APIs. The ESP32-CAM captures vehicle images and uploads them to the OpenALPR (Automatic License Plate Recognition) API for plate number extraction. The recognized license number can then be displayed on a serial monitor or web interface.

The main objectives of this project are:

- To design a low-cost, compact vehicle recognition system using ESP32-CAM.
- To capture and transmit real-time images via Wi-Fi to the OpenALPR cloud API.
- To decode the received OCR response and extract license plate text.
- To evaluate the recognition accuracy under various environmental conditions.

This system can be effectively used in smart parking, gated community entry systems, and toll booths to automate the vehicle identification process.

II. RELATED WORKS

License plate recognition has been a significant research area in computer vision and intelligent transportation systems. Early systems used standalone PCs with USB cameras and OpenCV for edge detection and segmentation. While accurate, these systems were not cost-effective for distributed deployment.

S. Patel et al. [1] demonstrated an Arduino-based vehicle logging system using RFID for entry control, which lacked visual recognition capability. K. Sharma et al. [2] developed a Raspberry Pi-based LPR system utilizing Tesseract OCR, achieving high accuracy but at a higher cost and power consumption.

The introduction of ESP32-CAM led to several compact IoT imaging systems. R. Gupta et al. [3] proposed an ESP32-CAM surveillance system with motion detection and cloud upload. However, their approach did not include OCR processing.

In this project, we integrate **ESP32-CAM's edge capture capabilities** with the **OpenALPR API**, enabling image recognition without needing heavy onboard processing. This hybrid (edge + cloud) approach ensures cost-effectiveness, scalability, and real-time response.

III. SYSTEM ARCHITECTURE AND DESIGN

The architecture of the proposed system consists of three layers: **Device Layer**, **Communication Layer**, and **Cloud Processing Layer**.

A. Hardware Components

- **ESP32-CAM:** Acts as the core processing unit with an inbuilt OV2640 camera sensor for capturing images and Wi-Fi for network communication.
- **FTDI Programmer:** Used for flashing and debugging via UART interface.
- **Power Supply:** A stable 5V source (via USB or external adapter) powers the module.
- **Optional Sensors:** An IR sensor or motion detector can trigger the image capture process automatically.

Connections:

- U0R → TX of FTDI
- U0T → RX of FTDI

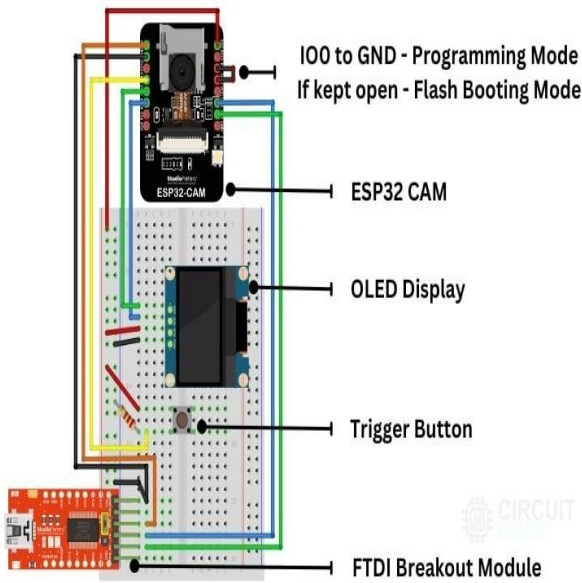


Fig. 1. System Connections of the ESP32-CAM License Plate Recognition System.

- GND → GND
- 5V → VCC
- IO0 grounded during upload for flashing

B. Software Components

- **Arduino IDE** with ESP32 board library for code development.
- **WiFi and HTTPClient Libraries** for network and API communication.
- **Base64 Encoding Library** for converting captured images into transferable formats.
- **OpenALPR API** for performing the cloud-based license plate recognition.

C. System Flow

- 1) Initialize the camera and Wi-Fi.
- 2) Capture an image of the vehicle using the ESP32-CAM.
- 3) Convert image data into a Base64-encoded string.
- 4) Send the image through an HTTP POST request to OpenALPR API.
- 5) Receive JSON response with recognized plate number and confidence score.
- 6) Display results via Serial Monitor or Web Server.

IV. SYSTEM IMPLEMENTATION

The system was implemented using the **ESP32-CAM** module programmed via **Arduino IDE**. The OpenALPR cloud API key was configured for authentication.

A simple web interface hosted by the ESP32-CAM displays the captured image and recognition output. The system can be triggered manually or automatically using motion sensors.

Firmware Workflow:

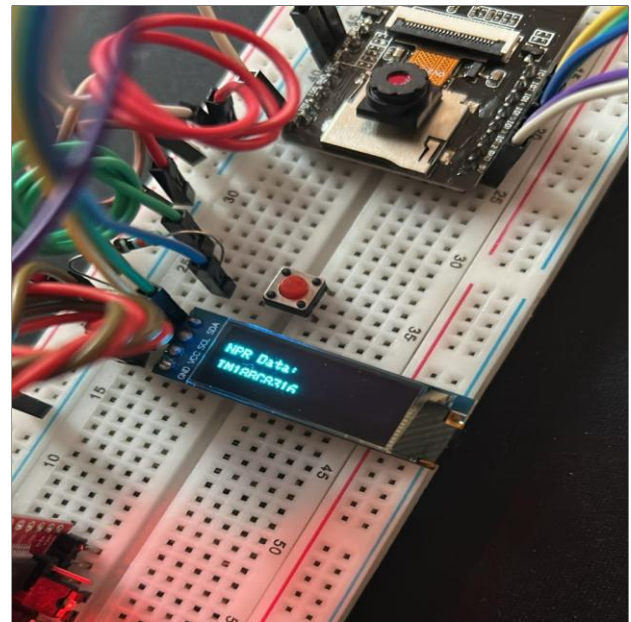


Fig. 2. Hardware Setup of the ESP32-CAM Prototype with FTDI Programmer.

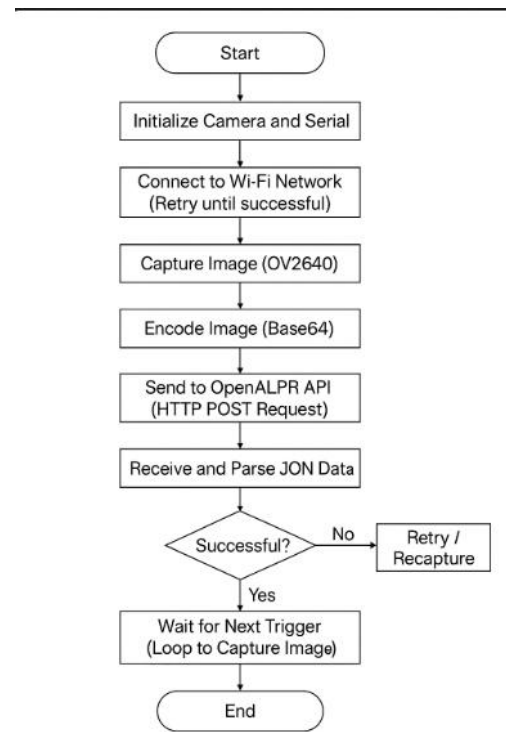


Fig. 3. Firmware Flowchart of the License Plate Recognition System.

- 1) Initialize camera configurations (frame size, pixel format).
- 2) Connect to Wi-Fi network.
- 3) Capture a frame using `esp_camera_fb_get()`.
- 4) Encode the frame and send an HTTP POST to `https://api.openalpr.com/v2/recognize_bytes`.
- 5) Parse JSON response to extract plate and confidence.
- 6) Display results on the serial terminal.

V. RESULTS AND DISCUSSION

The system was tested under various lighting conditions and camera distances. The OpenALPR API successfully recognized Indian license plates in most scenarios.

TABLE I
LICENSE PLATE RECOGNITION RESULTS

Scenario	Lighting	Distance	Accuracy
Daylight	1.5 m	Good	95%
Low Light	1.5 m	Moderate	85%
Night with Flash	1.5 m	Good	88%
Obstructed Plate	1.5 m	Poor	60%

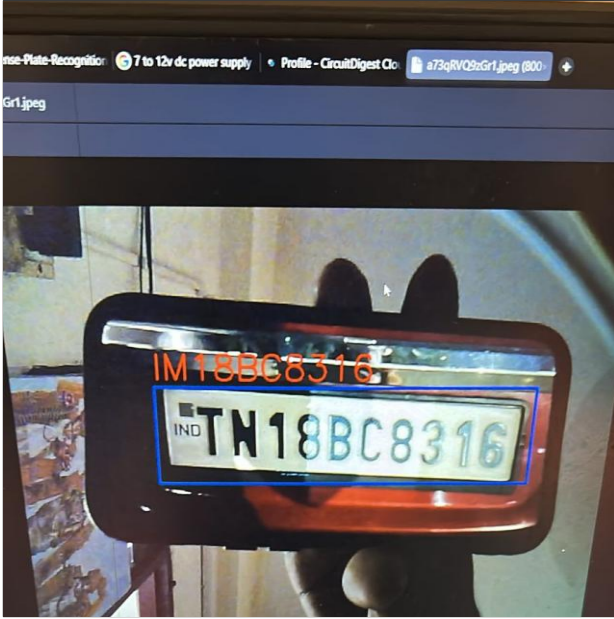


Fig. 4. Captured Image and Corresponding Plate Recognition Output.

Response time averaged **2.3 seconds** from capture to result display. The accuracy depended on the image clarity and viewing angle. The system proved efficient for controlled environments such as parking gates or campus security.

VI. CONCLUSION AND FUTURE WORK

This paper successfully demonstrates a **low-cost, IoT-based License Plate Recognition system** using ESP32-CAM and cloud integration. The system automates vehicle identification with real-time processing via the OpenALPR API.

It offers a scalable solution for smart parking, security monitoring, and automated toll collection.

Future Enhancements:

- Integrate local OCR models for offline recognition.
- Add servo-based tracking for moving vehicles.
- Store recognized numbers in a cloud database for analytics.
- Implement GSM connectivity for remote deployment.

REFERENCES

- [1] S. Patel, A. Kumar, "RFID-Based Vehicle Entry System Using Arduino," *IJERT*, vol. 9, no. 4, pp. 20–24, 2021.
- [2] K. Sharma, M. Jain, "Smart Vehicle Identification using Raspberry Pi and OCR," *IEEE IoT Conference*, 2020.
- [3] R. Gupta, V. Rao, "Smart Surveillance with ESP32-CAM and IoT," *IJCTE*, vol. 12, no. 5, pp. 45–49, 2022.
- [4] OpenALPR API Documentation, Available: <https://cloud.openalpr.com>.
- [5] Espressif Systems, "ESP32-CAM Datasheet," 2023.
- [6] Circuit Digest, "License Plate Recognition using ESP32-CAM," 2024.
- [7]